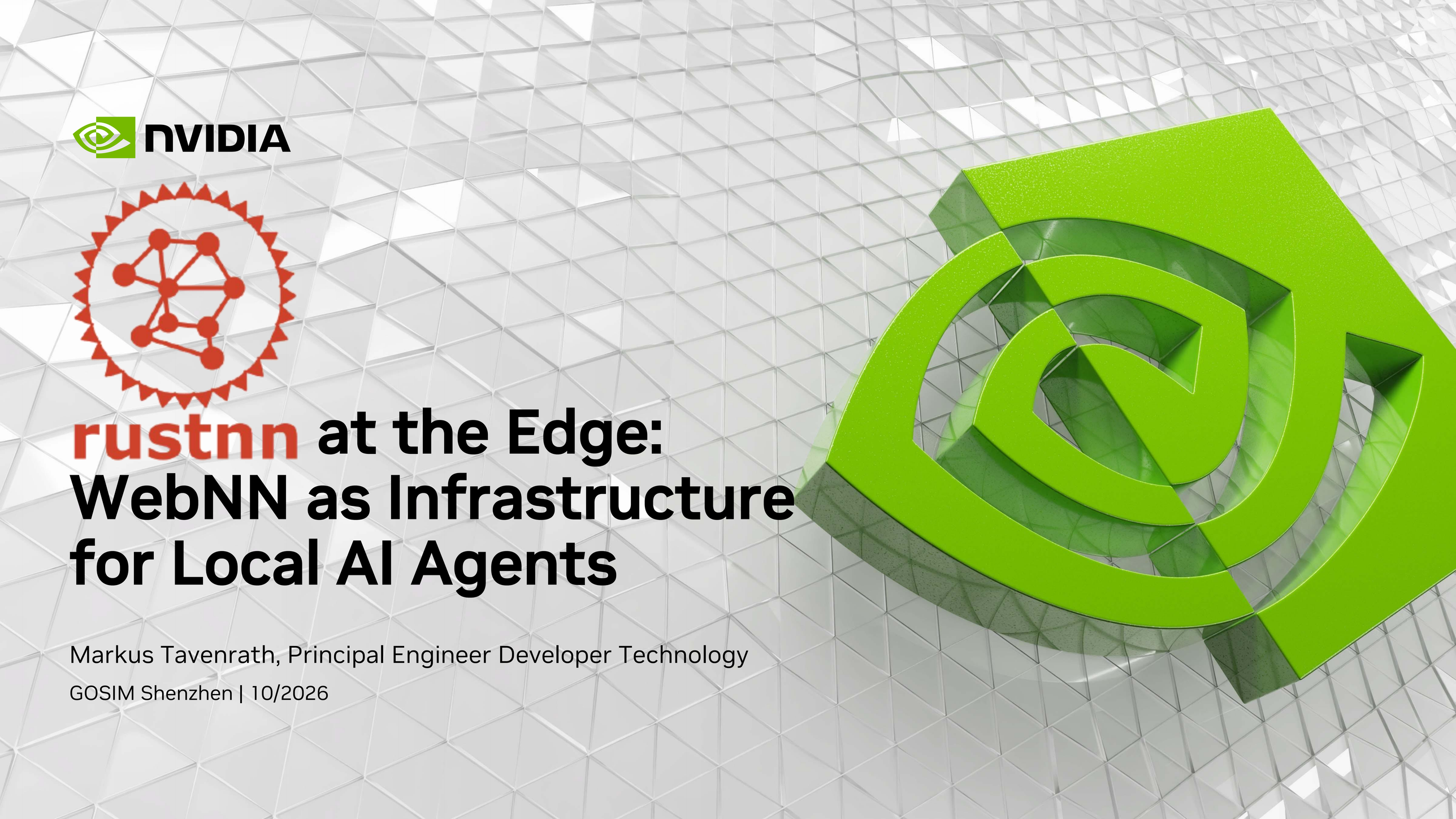


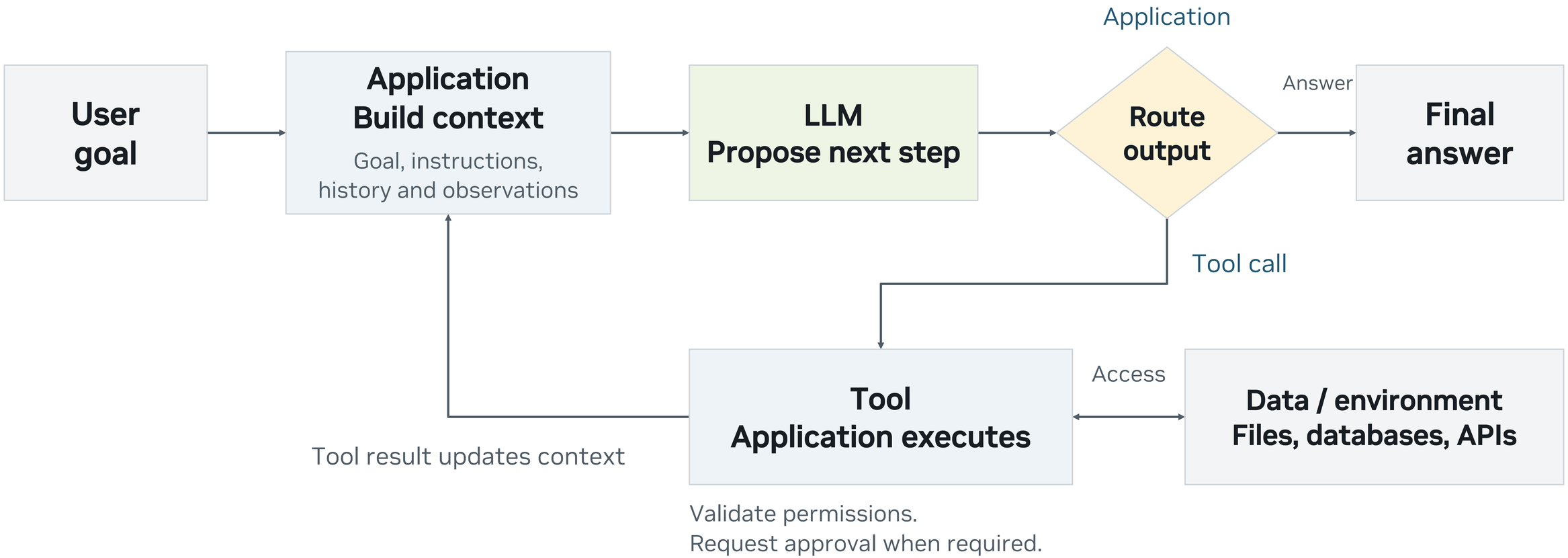
rustnn at the Edge: WebNN as Infrastructure for Local AI Agents

Markus Tavenrath, Principal Engineer Developer Technology

GOSIM Shenzhen | 10/2026

The agent loop

An LLM proposes an answer or tool call. The application manages execution



Tools access task data. Results become context for the next model call

Why run the whole loop locally?

Keep the task, its working context and its actions on the user's device

A private project assistant

"Read my project notes, transcribe this recording, inspect the screenshot, and draft a follow-up."

Retrieve, transcribe, inspect, draft, approve and save

Models, tools and task data can all stay on the device

Privacy	Documents, prompts and tool results can stay on the device
Offline	Work without connectivity after assets are installed
Responsiveness	Avoid network round trips between model calls and tools
Control	Choose models and keep state independent of an inference service
Economics	Use device compute without per-call cloud inference charges

The whole loop can operate without sending its working context to an inference service

NVIDIA

What does “local” mean?

Model inputs and inference results can stay on the user’s device

Native application	Rust, C++ or Python using a local inference interface
JS runtime	Node.js, Bun or another host with the required bindings
Wasm runtime	Portable application code inside a local sandbox
Browser	Local inference through browser APIs and cached model assets

Once assets are installed or cached, inference needs no remote service

NVIDIA

AI workloads inside an agent

The LLM makes decisions. Specialist models provide additional capabilities

LLM / text generation	Interpret the goal, propose actions and draft a response
Embeddings	Retrieve relevant context from local project notes
Image-to-text / vision	Read a screenshot, inspect a photo or recognize objects
Text-to-image	Generate a visual when the task calls for one
Speech-to-text / speech	Transcribe requests and produce spoken responses

A private project assistant can combine retrieval, vision and generation

NVIDIA

WebNN: a graph API for local inference

The W3C API describes computation; the implementation chooses how to execute it

Application

Define the same matrix graph used on slides 9–10

```
const a = builder.input('a',
  {dataType: 'float32', dimensions: [3, 4]});
const b = builder.input('b',
  {dataType: 'float32', dimensions: [4, 3]});
const c = builder.matmul(a, b);
const graph = await builder.build({c});
```

WebNN implementation

Validate and compile the graph

Run it on a supported backend

CPU **GPU** **NPU**

Hardware and operator support depend on the implementation

Same graph definition; execution follows the available device backend

NVIDIA

WebNN workloads in the browser

Examples from Microsoft's WebNN developer preview

Image generation



Stable Diffusion 1.5
Stable Diffusion Turbo
Stable Diffusion XL Turbo

Speech recognition



Whisper Base

Text generation



Phi-4-mini-instruct
TinyLlama-1.1B-Chat-v1.0
Qwen2-0.5B-Instruct
DeepSeek-R1-Distill-Qwen-1.5B

microsoft.github.io/webnn-developer-preview

NVIDIA

RustNN Ecosystem

A conformant WebNN implementation in Rust, with language bindings and hardware backends

WebNN Frontends

Application languages

- **Rust** — RustNN
- **Python** — PyWebNN
- **Node.js** — webnnjs ●
- **Browser** — Servo-ml ◆
- **C/C++** — RustNN

RustNN Core

Shared graph infrastructure

- **IDL Interface**
- **Graph Representation**
- **Validation**

Backends

Execution runtimes

- **ORT**
- **TRT-RTX**
- **CoreML**
- **Wasm WebNN**
- **LiteRT** ●
- **HiAI** ●
- **Burn** ◆

Conversion

Model and graph tools

- **torch2webnn** ◆
- **onnx2webnn**
- **webnn-graph**
  Includes webnn2js ●

● Work in progress (WIP)

◆ Proof of concept (POC)

NVIDIA

One WebNN graph in four languages

a: float32[3,4] × b: float32[4,3] → c: matmul(a, b)

JS · Browser / Node.js

```
const a = builder.input('a',
  {dataType: 'float32', dimensions: [3, 4]});
const b = builder.input('b',
  {dataType: 'float32', dimensions: [4, 3]});
const c = builder.matmul(a, b);
const graph = await builder.build({c});
```

Rust

```
let a = builder.input("a", &MLOperandDescriptor::new(
  MLOperandDataType::Float32, vec![3, 4]))?;
let b = builder.input("b", &MLOperandDescriptor::new(
  MLOperandDataType::Float32, vec![4, 3]))?;
let c = builder.matmul(a, b)?;
let graph = builder.build(
  &HashMap::from([("c", c)]))?;
```

Python

```
a = builder.input("a", [3, 4], "float32")
b = builder.input("b", [4, 3], "float32")
c = builder.matmul(a, b)
graph = builder.build({"c": c})
```

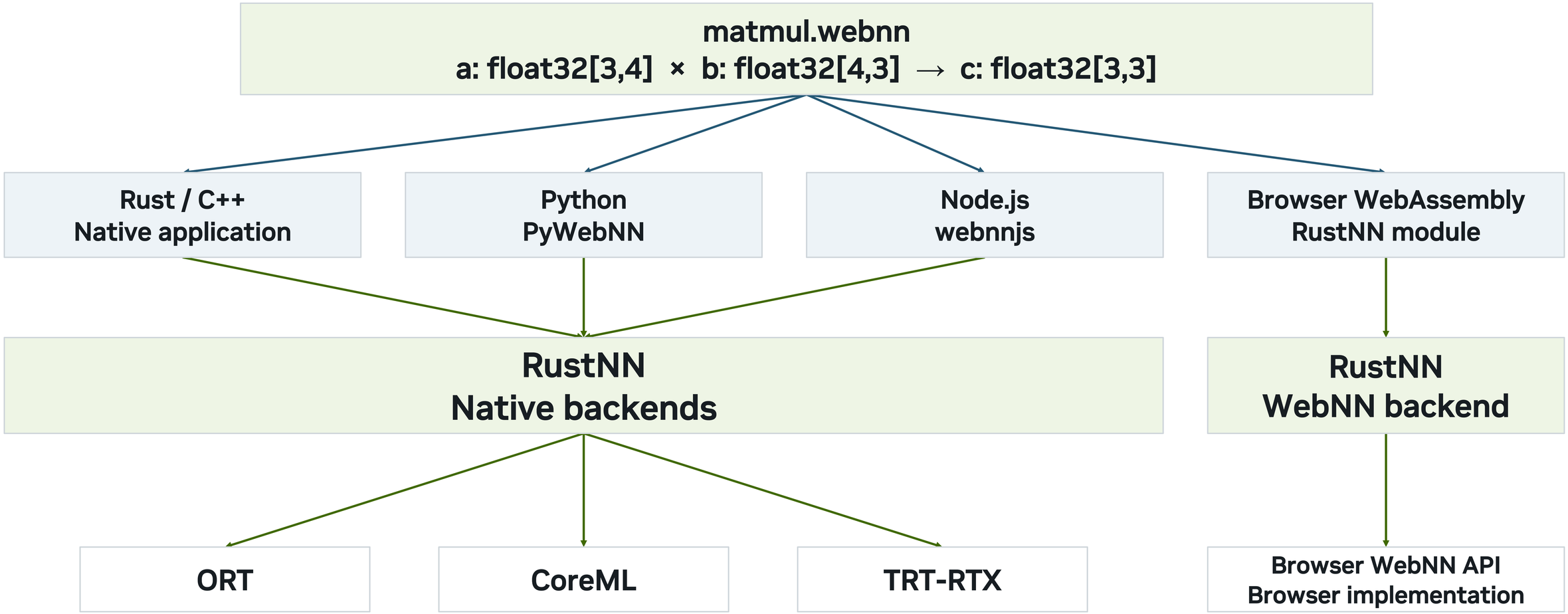
C++

```
auto a = builder.input("a",
  {rustnn::MLOperandDataType::Float32, {3, 4}});
auto b = builder.input("b",
  {rustnn::MLOperandDataType::Float32, {4, 3}});
auto c = builder.matmul(a, b);
auto graph = builder.build({{"c", &c}});
```


NVIDIA

One matrix graph across four application paths

The same c = matmul(a, b) graph reaches RustNN through four frontends



Native backends are alternatives selected for the host and workload.

NVIDIA

RustNN Conformance

WebNN Web Platform Tests across three native backends

rustnn



99.9%

pass rate for WebNN WPT tests
runnable outside a browser

Tested backends

ORT **TRT-RTX** **CoreML**

Browser-dependent tests are outside this result

WPT checks WebNN behavior. Model exports need separate numerical validation

NVIDIA

Transformers.js: workloads for agents

Task pipelines, tokenizers and generation logic for agent workloads

Workload	Agent capability
Text generation	Responses and proposed actions
Embeddings	Retrieval of local context
Vision / multimodal	Visual and document understanding
Speech	Spoken input and output

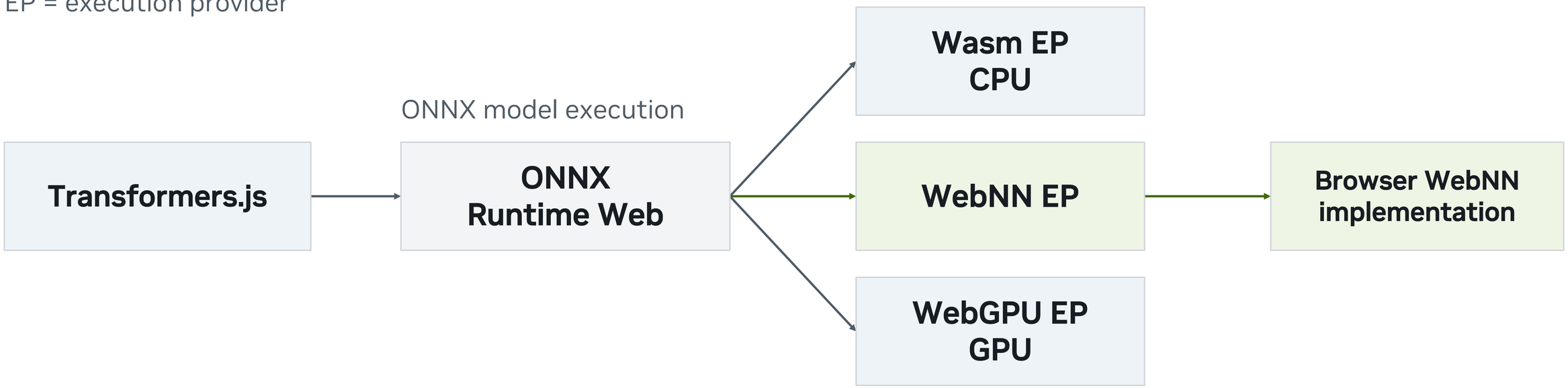
Existing ONNX model exports give onnx2webnn a workload collection to test

NVIDIA

Transformers.js execution paths

Three execution providers for browser inference

EP = execution provider



ORT Web can execute unsupported WebNN-path operations through its Wasm CPU EP.

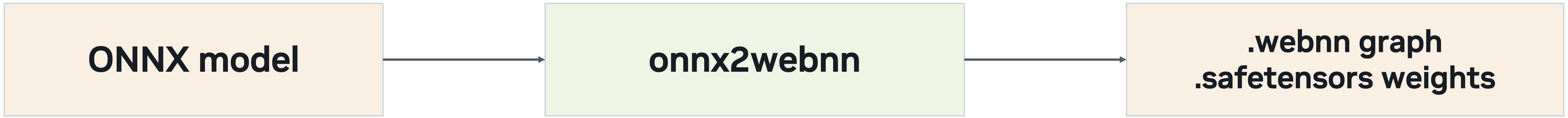
---

onnx2webnn prepares a direct WebNN path before deployment

NVIDIA

onnx2webnn: conversion before deployment

Export a supported model to a WebNN graph with separate tensor weights



Explicit compatibility

Lower supported operations and check shapes
Report gaps during model preparation

Numerical validation

Reload and execute the graph
Compare outputs with the source model

The file format belongs to this ecosystem. WebNN standardizes the graph API

NVIDIA

Numerical validation by Transformers.js model

Real-weight sweep, 22 September 2026. Grouped by model repository

25 / 28

model groups pass
all selected checks

3 groups with blockers

FastVLM-0.5B-ONNX
Voxtral-Mini-3B-2507-ONNX
chronos-2-onnx

Model family	Pass / total
Text and retrieval	11 / 11
Vision and video	6 / 6
Vision-language	5 / 6
Speech and audio	2 / 3
Music generation	1 / 1
Time-series forecasting	0 / 1

47/51 configured checks across 44 ONNX files. Selected components only.
Fixed input and cache shapes; numerical comparison on ORT CPU.

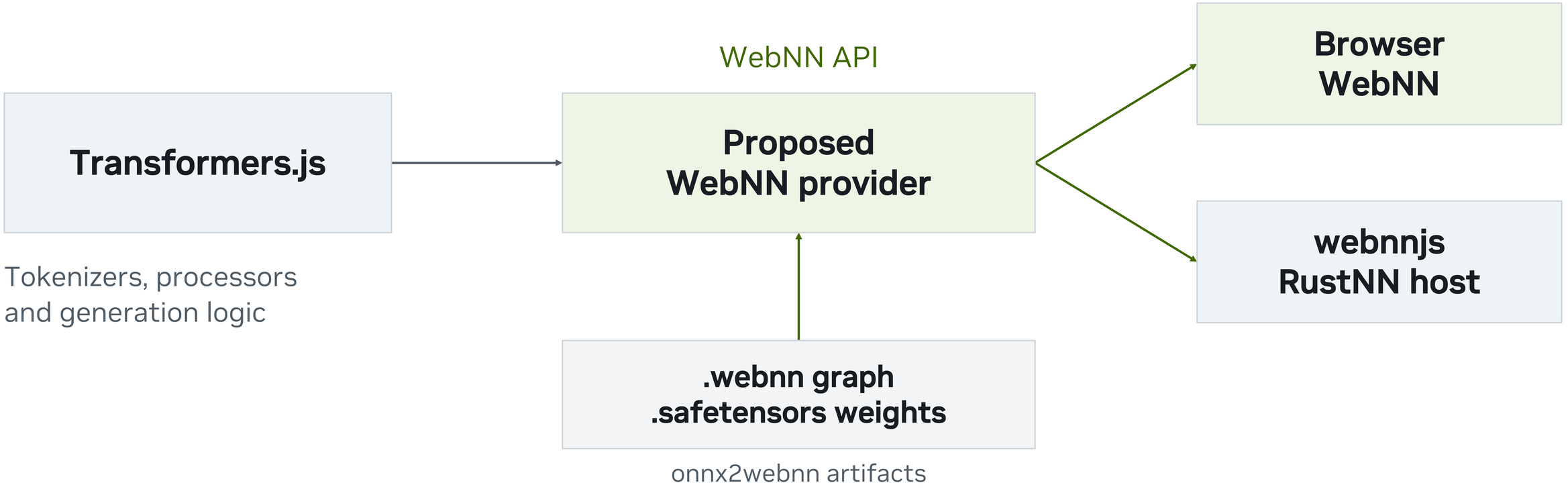
Complete task pipelines and growing KV-cache reuse need separate validation

NVIDIA

Exploring direct WebNN with Transformers.js maintainers

A proposed integration path that helps motivate our onnx2webnn work

PR #1732: pluggable inference providers (draft)

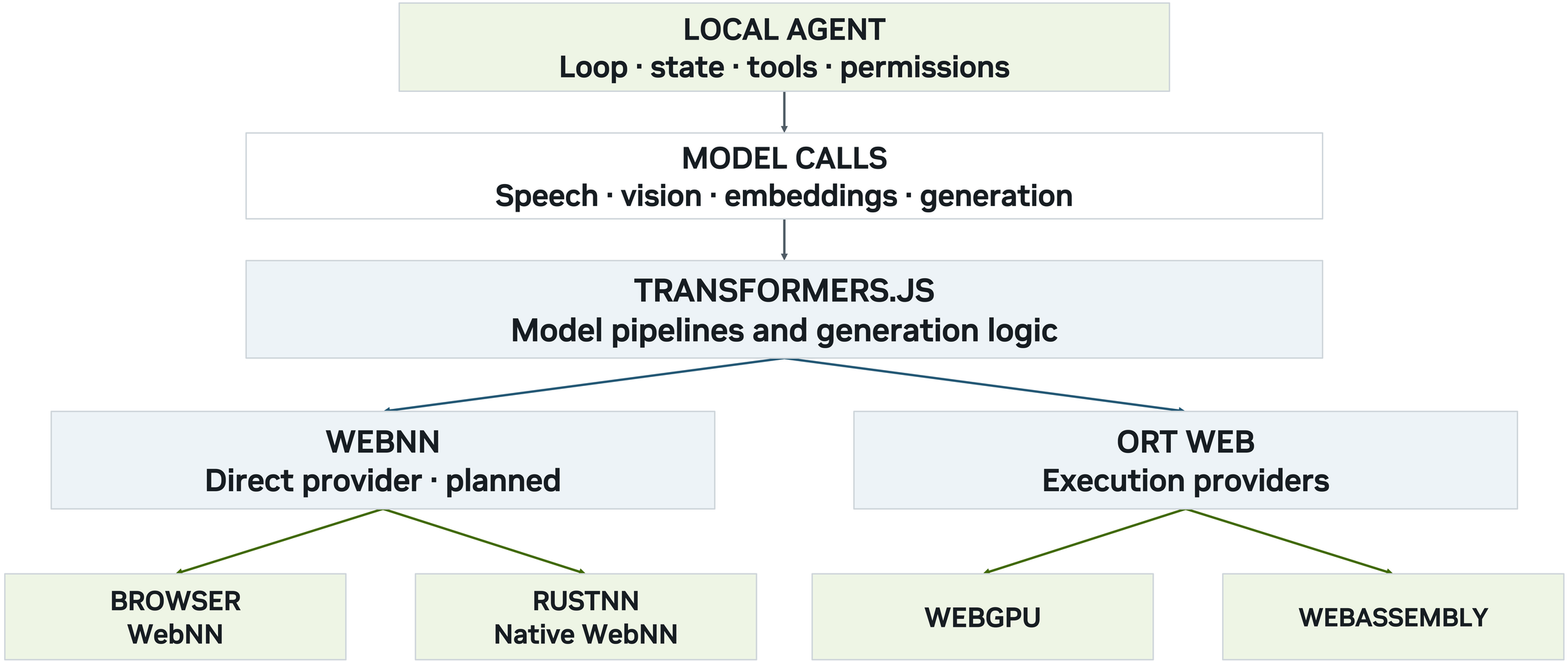


Converted WebNN graphs could run directly while Transformers.js keeps its generation logic

NVIDIA

Desired local agent inference flow

Direct WebNN support in Transformers.js is planned



NVIDIA

What's next for RustNN?

Conformance, browser integration, GPU execution and model coverage

Conformance

Run the complete WPT suite in a real web environment.

Browser integration

Update the Firefox prototype to gain tests and benchmarks.

GPU backend

Explore Burn WebGPU and ORT WebGPU, including subgroup matrix support.

Content and reliability

Export more models and harden converters and backends.

GPU backend discussion: github.com/rustnn/rustnn/pull/135

NVIDIA

Thank you

Questions?

RustNN

github.com/rustnn/rustnn

onnx2webnn

github.com/rustnn/onnx2webnn